

Eliminating the Decision-Feedback Loop: A Forgetting-Rate-Sized Feed-Forward Substitute at 1/59 the Standard-Cell Area

Second draft. Sections II–IV develop the model, Section V the circuit that creates the constraint, Sections VI–VIII the measurements, Section IX the RTL, and Section X the experiments that follow and the conditions under which we stop.

Abstract—Receiver design has moved, generation by generation, out of the analog front end and into a digital datapath: 25 Gb/s per lane became 112, then 224, and OIF is now framing 448, while 5G and 6G fronthaul rides the same electrical lanes. That migration made the receiver a large digital design-space problem, and high-level synthesis (HLS) became the vehicle for exploring it—successfully, for the feed-forward parts, which HLS pipelines to an initiation interval (II) of one. Receivers, however, are not feed-forward. Their inner loops—decision feedback, add-compare-select, timing and gain recovery—are recurrences with a *quantizer* inside the loop, and the recurrence, not the arithmetic, fixes the achievable II. The classical remedy, look-ahead, is exact for linear recurrences and fails here, because a slicer does not commute with the algebra look-ahead performs; what remains is speculation, whose cost is exponential in the depth unrolled. The HLS literature shortens or reorders such recurrences; none removes one. We remove one, and we size the removal rather than guess it. Modelling the loop as an input-driven finite automaton, we define its *forgetting rate* $p(W)$ —the probability that two copies driven by the same input have not yet coupled after W steps—and show that the replacement error of a feed-forward substitute reading W quantized inputs is bounded by $p(W) + \epsilon_q(B)$, the second term being the probability that B -bit quantization alone flips a decision. Both terms are measurable before any RTL is written. On an 8-tap decision-feedback equalizer carrying the whole postcursor of a 20 dB channel—removing the loop raises the bit-error rate by 1363 $\times$, so the loop is genuinely working—we measure p decaying geometrically at $\lambda = 0.338$ per symbol, giving $W(10^{-3}) = 5$. Synthesizing the substitute to a 130 nm open standard-cell library, a $B = 4$, $W = 2$ instance occupies $142.6 \mu\text{m}^2$ in 26 gates against $8395.6 \mu\text{m}^2$ in 924 gates for the recurrent equalizer—**58.9 $\times$ less area, with no recurrence edge at all**—while its bit-error rate, 8.89×10^{-5} , is that of the exact recursion and clears the KP4 RS(544,514) threshold we derive here (2.2619×10^{-4}). Two further results change how such a substitute should be costed and sized. First, the natural cost model is wrong: a 65 536-entry map synthesizes to 816 gates and a 1 048 576-entry map to 3537, because logic minimization exploits the fact that only 1.4 % of the address space is ever visited; measured gate count tracks *occupied* entries, at ≈ 0.23 gates each, not address width. Second, and counter to intuition, *widening the address makes the design worse*: at 20 address bits the bit-error rate degrades to 8.33×10^{-3} , a hundredfold, because the map no longer fills. The binding resource is training data, not address space, and that gives the transformation an interior optimum which p and ϵ_q together predict. We give the automaton model, the circuit-level origin of the one-unit-interval constraint measured in the same technology, synthesizable SystemVerilog for all three forms, and the FPGA experiment—with its controls and its kill conditions—that the claims still require.

Index Terms—High-level synthesis, initiation interval, loop-carried dependence, decision-feedback equalization, finite automata, coupling, forward error correction, standard-cell area, Reed–Solomon.

I. INTRODUCTION

This paper is about one sentence a synthesis tool prints, and what it costs: *cannot pipeline loop: carried dependence*.

Two decades of rate scaling changed what a receiver is. At 25 Gb/s per lane it was a continuous-time equalizer, a slicer, and a short feedback tail. At 112 Gb/s it is an analog front end, a time-interleaved converter, and a digital signal processor; receivers of that class spend roughly half their analog power on the converter and interleave it 64 ways. At 224 Gb/s the same architecture compensates 31 to 50 dB of loss, and OIF's CEI-448G framework now frames a generation whose modulation is still under discussion [23]. Wireless fronthaul rides the same lanes: an O-RAN 7-2x split carries frequency-domain samples over Ethernet, and the electrical interface underneath is the same one.

The design consequence is that a receiver is now a large digital parameter space—tap counts, word lengths, detector memory, the interaction with the outer code—too large to explore in hand-written RTL. High-level synthesis is what the field reached for, and for feed-forward datapaths it works: a C loop over a finite impulse response is scheduled, pipelined and retimed to an initiation interval of one, and the designer explores by editing constants.

A. What an initiation interval is, and why a loop sets it

A pipelined circuit starts a new computation every II cycles. II = 1 means one result per clock, which is what a line-rate datapath needs. A pipeline achieves this by cutting a long combinational path into stages separated by registers, so the clock period need only cover one stage.

That trick requires the computation to flow forward. If iteration n needs a value produced by iteration $n-1$, no register may be inserted between them without delaying the value past the point where it is needed. For a loop whose dependence path spans d pipeline stages, no new iteration may begin until d cycles have elapsed, and the

recurrence-constrained minimum initiation interval obeys $\text{RecMII} \geq d$ [9]. The arithmetic is not the problem; the *cycle in the dependence graph* is.

Table I lists the loops at the centre of a receiver. They share a shape: a state updated from its own past, with a quantizer—a slicer, a sign, a minimum, a saturation—inside the cycle.

TABLE I

Nonlinear recurrences at the centre of a receiver. Each is RecMII-bound, and none is reachable by algebraic look-ahead, because a quantizer sits inside the cycle.

Kernel	Recurrence	In-loop nonlinearity
DFE	$d[n] = Q(x[n] - \sum c_k d[n-k])$	slicer Q
Viterbi ACS	$M[n] = \min_j (M_j[n-1] + \lambda)$	minimum
Bang-bang CDR	$\phi[n] = \phi[n-1] + \mu \text{sgn}(e)$	sign
AGC	$g[n] = g[n-1] + \mu \text{sgn}(y - r)$	sign
Layered min-sum	row-wise message update	min, sign
DD-LMS	$\mathbf{w}[n] = \mathbf{w}[n-1] + \mu \mathbf{e} \mathbf{x}$	decision device

B. Why the classical remedy stops at the quantizer

For a *linear* recurrence, look-ahead removes the bottleneck exactly. Substituting $y[n] = ay[n-1] + x[n]$ into itself gives $y[n] = a^2y[n-2] + ax[n-1] + x[n]$, which halves RecMII because the dependence now reaches back two iterations instead of one; scattered look-ahead extends this to arbitrary depth at logarithmic overhead [1], [2].

The substitution needs the loop operator to compose through the same algebra. A quantizer does not: $Q(aQ(u) + v)$ cannot be rewritten as an affine function of u and v . Every row of Table I is therefore out of look-ahead's reach. What remains is *speculation*—precompute the datapath for every value the in-loop nonlinearity can take, and select afterwards—which is the loop-unrolled decision-feedback equalizer

[7] and the M -step add-compare-select of the parallel Viterbi algorithm [3]. It is exact, and it costs $|A|^d$ copies of the speculated datapath.

C. What we do

We remove the recurrence instead, and size the removal from a measurement rather than a guess. The three forms are drawn in Fig. 1. Our contributions are:

- A model of the loop as an input-driven finite automaton, and a *forgetting rate* $\rho(W)$ defined by coupling, with a two-term bound $\rho(W) + \epsilon_q(B)$ on the replacement error of a feed-forward substitute reading W inputs quantized to B bits (Section IV).
- The measurement, on a loop verified to be doing the work ($1363 \times$ bit-error-rate penalty when removed): $\lambda = 0.338$ per symbol, $W(10^{-3}) = 5$ (Section VI).
- Synthesis of both forms into the same 130 nm library: $142.6 \mu\text{m}^2$ and 26 gates against $8395.6 \mu\text{m}^2$ and 924 gates, at equal bit-error

rate and inside the KP4 threshold (Section VII).

- Two corrections to how such a substitute is costed and sized: synthesized area tracks *occupied* map entries rather than address width, and widening the address degrades the design because the map stops filling (Section VIII).
- Synthesizable SystemVerilog for all three forms (Section IX), and the FPGA experiment, controls and kill conditions that the claims still require (Section X).

Is this a mathematical or an engineering contribution? An engineering one, and we say so plainly. Coupling arguments, substochastic spectral radii and Markov models of decision-feedback error propagation are standard machinery, and we claim none of them. What is new is the use: turning error propagation—normally a performance nuisance to be bounded—into a *compile-time sizing parameter* for a transformation that removes a dependence cycle, and pricing the result in standard-cell area and in decibels at the code's threshold rather than in numerical error. The two cost corrections of Section VIII are empirical findings with mechanisms, not theorems.

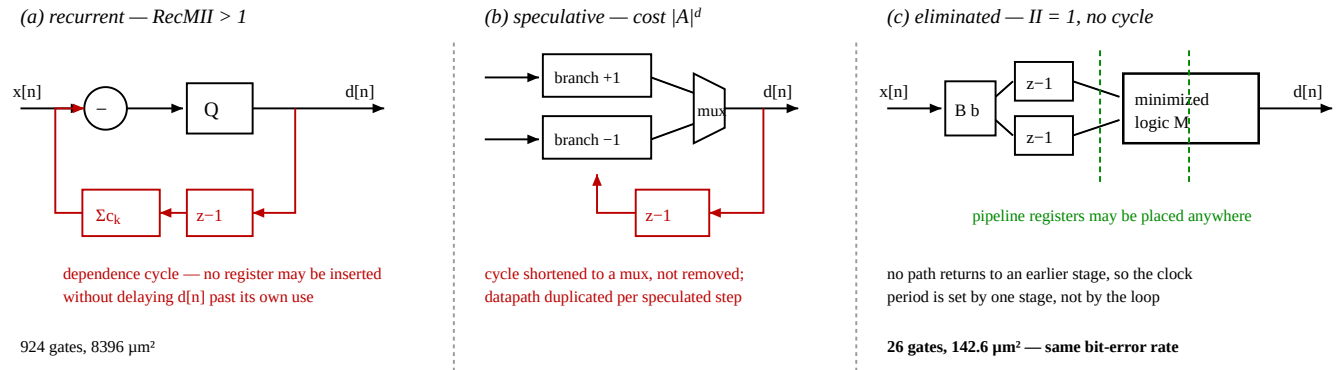


FIG. 1. Three implementations of the same 8-tap decision-feedback equalizer. (a) The recurrent form: the slicer output must return to the summing node before the next symbol, so no register may be placed in the red path and $\text{RecMII} > 1$. (b) Speculation precomputes both branches and shortens the cycle to a multiplexer, at the cost of duplicating the datapath once per speculated step. (c) The eliminated form reads a window of quantized inputs and has no cycle at all, so registers may be placed freely and $\text{II} = 1$ at any clock the logic depth permits. Areas are measured, Section VII.

II. RELATED WORK

A. Recurrence-bound II in HLS

The HLS literature attacks RecMII by scheduling and by architecture. Loop splitting separates iterations whose dependences differ so the common case pipelines [11]; polyhedral analysis admits dynamic pipelining where the dependence distance is data-dependent [12]; dynamic scheduling and its static hybrids resolve dependences at run time with load-store queues and rollback [10]; speculative loop pipelining models the paths explicitly [14]; dynamic loop fusion handles loops whose dependences cross [13]. On the architecture side, recurrence-aware placement collapses a multi-stage recurrence path into few registered cycles by co-locating its endpoints [9]. Each of these *shortens* or *reorders* the cycle. None removes it, and none is specific to the case where the loop operator is a quantizer with a small output alphabet—which is the case that admits removal.

B. Look-ahead, unfolding, retiming

Parhi's transformations are the exact remedy for linear recurrences and the standard reference for pipelining recursive filters [1], [2]. Their scope is what we build on rather than a shortcoming: the algebra is exact *because* the loop is linear.

C. Domain-specific speculation and state reduction

The loop-unrolled decision-feedback equalizer [7] and the parallel Viterbi algorithm that breaks the add-compare-select bottleneck by M -

step look-ahead [3] are the canonical instances of exhaustive enumeration in receivers. Reduced-state and delayed-decision sequence estimation [27], [28] cut the state count by merging states—a *structural* approximation. Our forgetting rate is its *temporal* counterpart: it does not merge states, it establishes that old states cease to matter. The two should compose, and testing that is in Section X.

D. Tables in place of computation

Distributed arithmetic replaces multiplication by lookup [4], but the stored function is linear in the address and the technique does not touch recurrences. Data-dependent noise-predictive detection in magnetic recording conditions a branch metric on the data pattern [5], [6]—the nearest prior instance of "condition on a window instead of computing"—and is realized as a bank of filters, one per pattern. The distinguishing question here is not whether to tabulate, but *how wide the window must be*, which is what ρ answers.

E. Error metrics for approximation

Approximate HLS judges a transformation by average error, maximum error, mean absolute percentage error, peak signal-to-noise ratio, or bit-error rate [18]. The first four are statistics of error magnitude; none, including the last, is a statistic of the error's temporal correlation, which an outer code is acutely sensitive to (Section VIII-C). We therefore report the substitute's own bit-error rate against the code's threshold rather than a numerical distance, and record the error positions so that the post-decoding evaluation is available.

III. THE LOOP AS A FINITE AUTOMATON

Write the decision-feedback recurrence with its state made explicit. Let $A = \{-1, +1\}$ be the slicer alphabet, $\mathbf{c} = (c_1 \dots c_K)$ the tap vector, and

$$\mathbf{s}[n] = (d[n], d[n-1], \dots, d[n-K+1]) \in A^K \quad (1)$$

the state. The update is

$$d[n] = Q(x[n] - \mathbf{c}^T \mathbf{s}[n-1]), \quad Q(u) = \text{sgn}(u). \quad (2)$$

Two observations follow immediately, and they are the whole basis of the method.

The state set is finite. $|A^K| = 2^K$; for the 8-tap equalizer we study it is 256. The loop is therefore a *deterministic finite automaton driven by a real-valued input*: a fixed input sequence maps each starting state to a definite trajectory.

The automaton may forget. Drive two copies of (2) from different initial states with *the same* input sequence. If at some step they produce the same decision, their states become identical and—because the map is deterministic and the input is shared—*remain* identical forever. In the language of Markov chains the two copies have *coupled*. Fig. 2 shows what this looks like.

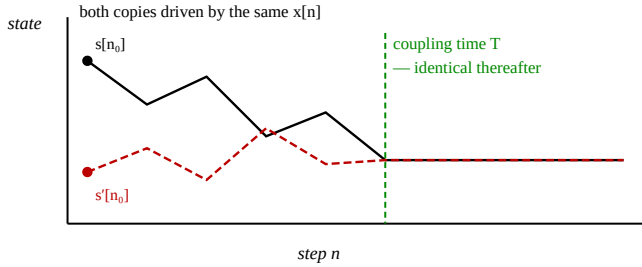


FIG. 2. Two copies of the loop started in different states and driven by the same input. Because the update is deterministic and the input is shared, once the trajectories agree they agree forever. The forgetting rate is the probability that this has not yet happened.

A. Forgetting rate

Definition 1. Let T be the coupling time of two copies of (2) started at time n_0 in states differing by one admissible perturbation and driven by a common input. The *forgetting rate* is

$$\rho(W) = \Pr\{d[n_0+W-1] \neq d'[n_0+W-1]\} \leq \Pr\{T > W\}. \quad (3)$$

If the input process were independent and identically distributed, the pair $(\mathbf{s}[n], \mathbf{s}'[n])$ would be a Markov chain on $A^K \times A^K$, absorbing on the diagonal, and standard theory would give

$$\Pr\{T > W\} \leq C \lambda^W, \quad (4)$$

with λ the spectral radius of the transition matrix restricted to the off-diagonal states. We use (4) as the functional form and estimate λ by regression on measured ρ , because the input to a real equalizer carries residual intersymbol interference and is therefore not independent across symbols; a bound computed from an assumed independent input would be a bound on a different system. The geometric form is what we verify, not assume: Section VI reports the fit.

B. The substitute and its two error terms

Let q_B be a uniform B -bit quantizer of step Δ , and define the feed-forward map

$$M: (q_B(x[n]), \dots, q_B(x[n-W+1])) \rightarrow A. \quad (5)$$

Proposition 1 (replacement error). The substitute (5) disagrees with the exact recursion (2) with probability at most

$$\varepsilon \leq \rho(W) + \varepsilon_q(B), \quad \varepsilon_q(B) = E_u[(\Delta/2 - |u|)_+ / \Delta], \quad (6)$$

where $u = x[n] - \mathbf{c}^T \mathbf{s}[n-1]$ is the slicer input.

Argument. The substitute can err in only two ways, and a union bound covers them. Either (i) the state at $n-W$ still influences $d[n]$, in which case no function of the last W inputs can be correct for every state, and this has probability at most $\rho(W)$ by Definition 1; or (ii) the state is irrelevant but the quantization of the window moves the slicer input across zero. For rounding error e uniform on $[-\Delta/2, \Delta/2]$, the sign flips exactly when $|e| > |u|$ with opposing sign, whose probability conditioned on u is $(\Delta/2 - |u|)_+ / \Delta$; averaging over u gives the second term. $\square$

This is the practical content of the model. **Both terms are measurable before any RTL exists:** $\rho(W)$ from a perturbation experiment on the reference implementation, and $\varepsilon_q(B)$ from the distribution of the slicer input, which a designer already has. The design rule is to choose the smallest (B, W) whose bound meets the tolerance, and Section VIII shows why "smallest" rather than "largest" is the right direction.

What Proposition 1 does not say. It bounds the substitute's disagreement with the *exact recursion*, not its bit-error rate, and the two differ: a disagreement is harmful only if the exact recursion was right. In our measurements the substitute's bit-error rate is close to ε because the exact recursion's own error rate is an order of magnitude smaller, but that is a property of the operating point, not a theorem. We also take the perturbation in Definition 1 to be a single most-recent-decision flip; whether that is the worst case over admissible state pairs is open (Section XI).

IV. THE CIRCUIT THAT CREATES THE CONSTRAINT

RecMII is a statement about a dependence graph, but in a receiver it has a physical origin, and it is worth seeing at transistor level because that is where the "one unit interval" comes from. Fig. 3 draws the first tap of an analog decision-feedback equalizer. Every number quoted in this section was measured by transient simulation of these netlists in a 130 nm open process design kit, with the device models the kit ships.

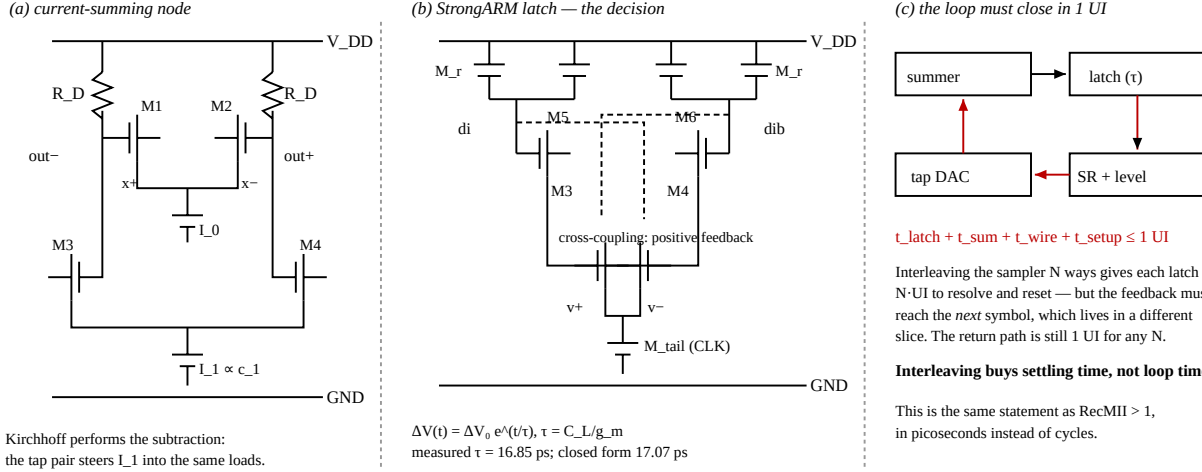


FIG. 3. The first decision-feedback tap at device level. (a) The summing node is a differential pair loaded by R_D ; the tap is a second pair steering $I_1 \propto c_1$ into the same loads, so Kirchhoff performs the subtraction: the tap pair steers I_1 into the same loads. (b) The StrongARM latch resolves by regeneration: the tail transistor switches on with the clock, the input pair converts the differential input into an initial imbalance ΔV_0 , and the cross-coupled pair amplifies it exponentially with time constant $\tau = C_L/g_m$; four reset devices precharge the internal nodes while the clock is low, so the circuit draws no static current. (c) The closed path summer $\rightarrow$ latch $\rightarrow$ level shift $\rightarrow$ tap must complete within one unit interval.

A. What the devices say

Three measurements from this technology matter for what follows.

The regeneration time constant, and where its capacitance lives.

Biasing the cross-coupled stage at its inversion point gives $g_m = 4.699$ mS. Breaking the loop and measuring the node capacitance including the next stage's gate gives $C_L = 80.2$ fF—the external load is 2 fF, so the capacitance is almost entirely the cross-coupled pair's own gates. The closed form then gives $\tau = 17.07$ ps against 16.85 ps measured by sweeping the input amplitude over three decades and fitting $t_{\text{resolve}} = t_0 + \tau \ln(1/\Delta V)$, a 1.3 % agreement. The formula was never in doubt; what was wrong in our first attempt was the capacitance, by a factor of 6.7, because only the external load had been counted.

Interleaving does not help the loop. A receiver cannot clock one latch at line rate and interleaves the sampler N ways; published 112 Gb/s parts use $N = 64$. Each latch then has $N \cdot \text{UI}$ to resolve and reset. But the feedback must reach the *next* symbol, which belongs to a different slice, so the path from slice k 's decision to slice $k+1$'s summing node remains one unit interval for any N . Interleaving buys settling time for the sampler and nothing for the loop. This is exactly why high-rate equalizers unroll rather than interleave, and it is the physical reason the transformation of this paper is worth having.

A measurement discipline that this section inherited. Two early numbers here were wrong in ways worth naming, because Section VIII repeats the pattern in a digital setting. Measuring the latch's resolution on a single node returned a 47.3 ps plateau that no tightening of solver tolerances moved; driving the differential input to zero showed the latch remaining metastable while the measurement still returned a time—it had been following the common mode. Measuring it differentially removed the artifact. Separately, reading the driver's output swing with a windowed maximum gave a value 30 % above the ideal $2I_{\text{eff}}$, because the window caught the input edge's feedthrough spike; reading at a settled instant gave a ratio of 1.000 against the tail current measured through a separate sense element. In both cases the instrument, not the circuit, was the source of the anomaly.

V. METHOD

The transformation is four steps, and each produces a number that can be checked.

FORGETTING-RATE ELIMINATION (FRE)

1. **Verify the loop earns its place.** Run the reference with and without the recurrence. If removing it does not degrade the metric, the loop is idle and nothing needs eliminating—the transformation would appear to succeed for the wrong reason.
2. **Measure $p(W)$** by the perturbation experiment of Definition 1 and fit (4) to obtain λ . Read off $W(\epsilon)$.
3. **Compute $\epsilon_q(B)$** from the slicer-input distribution by (6), and pick the smallest (B, W) meeting the tolerance.
4. **Learn and emit.** Populate M on the training prefix, emit it as a truth table, and let logic synthesis minimize it. Report area, gate count, the substitute's own error rate, and its position relative to the code's threshold.

Step 1 is not a formality. On our first attempt we measured a substitute at $50 \mu\text{m}^2$ in 7 gates with *zero* disagreement, which we nearly reported. The operating point had an 11-tap feed-forward equalizer ahead of the loop, leaving the largest feedback tap at 0.041; the loop was doing nothing, the correct answer was a plain slicer, and the synthesizer had found it. The result was real and meant nothing. Every measurement below is taken at an operating point where removing the loop raises the bit-error rate by $1363\times$.

Setup. A 20 dB channel, no continuous-time equalizer, a single feed-forward tap, and an 8-tap decision-feedback equalizer adapted by least mean squares, at 30 dB signal-to-noise ratio; 600 000 symbols, the first 40 % for adaptation and map training and the remaining 360 000 held out for every number reported. The adapted taps are (0.589, 0.359, 0.253, 0.192, 0.151, 0.128, 0.107, 0.095), energy 0.636. The reconstruction of the exact recursion from the reference sampler output reproduces the reference bit-error rate to 8.89×10^{-5} against 9.52×10^{-5} ; we check this before using it, because a reconstruction that does not reproduce its reference is measuring a different system. Synthesis is to a 130 nm open standard-cell library through an open flow, with cell area read from the library's LEF SIZE record and device counts from its SPICE view; the flow refuses to report an area if any mapped gate is unmatched, since a silently dropped gate biases area downward, which is to say toward the author's thesis.

VI. MEASURED FORGETTING RATE

Fig. 4 shows $p(W)$ over 8000 independent perturbation trials.

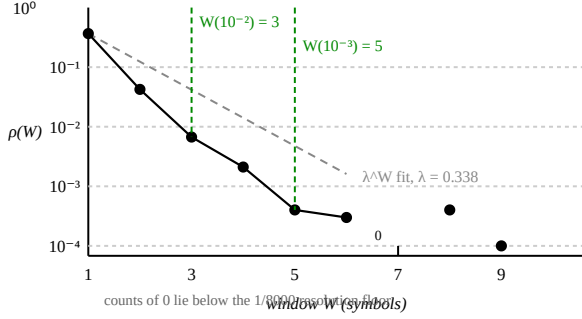


FIG. 4. Measured forgetting rate of the 8-tap loop. A single flipped decision is still visible one symbol later with probability 0.365, and after five symbols with probability 4×10^{-4} . The decay is geometric at $\lambda = 0.338$ per symbol over the range where the estimator has resolution; the tail falls slightly below the fit, so using the fit to size W is conservative. Points at zero are plotted at the resolution floor of $1/8000$.

The loop forgets quickly: $W(10^{-1}) = 2$, $W(10^{-2}) = 3$, $W(10^{-3}) = 5$. A window of five symbols therefore suffices to make the state at $n-5$ irrelevant to the decision at n all but one time in 2500—this for a loop whose removal costs three orders of magnitude of bit-error rate. That combination, a loop that matters and a state that does not persist, is exactly the regime the transformation is for.

VII. SYNTHESIS RESULTS

Table II reports every measured instance. The recurrent reference, generated from the same tap values, occupies $8395.6 \mu\text{m}^2$ in 924 gates and 9188 devices.

TABLE II

Eliminated substitutes against the recurrent reference, all synthesized to the same 130 nm library. Bit-error rate is on 360 000 held-out symbols; the exact recursion scores 8.89×10^{-5} and the KP4 threshold is 2.2619×10^{-4} .

B	W	addr	entries	occupied	disagree	BER	μm^2	gates	vs. loop
4	2	8	256	112	0.018 %	$8.89\text{e-}5$	142.6	26	58.9×
5	2	10	1024	362	0.016 %	$7.22\text{e-}5$	568.0	87	14.8×
6	2	12	4096	1225	0.029 %	$2.22\text{e-}4$	1182.4	182	7.1×
4	3	12	4096	620	0.026 %	$1.67\text{e-}4$	1331.3	202	6.3×
5	3	15	32768	2801	0.109 %	$1.02\text{e-}3$	4330.4	674	—
4	4	16	65536	2655	0.109 %	$1.00\text{e-}3$	5052.3	816	—
6	3	18	262144	13882	0.638 %	$6.33\text{e-}3$	14995.6	2351	—
5	4	20	1048576	14830	0.840 %	$8.33\text{e-}3$	22058.7	3537	—

Rows above the rule clear the threshold; rows below do not.

The headline instance is $B = 4$, $W = 2$: $142.6 \mu\text{m}^2$ in 26 gates, disagreeing with the exact recursion on 0.018 % of symbols, with a bit-error rate of 8.89×10^{-5} —the exact recursion's own—and no recurrence edge. Against the recurrent form that is **58.9× less cell area**, and the II constraint that motivated the work is gone by construction rather than by scheduling effort.

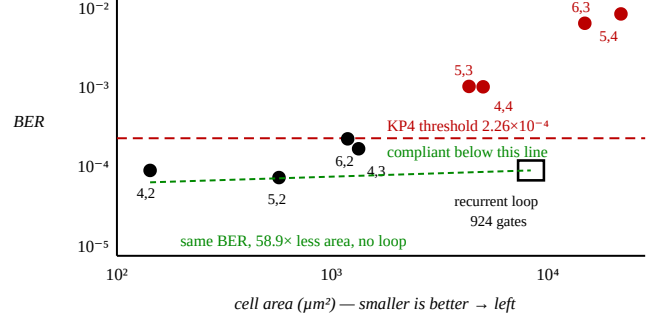


FIG. 5. Each substitute placed by its cell area and its bit-error rate, with the recurrent reference as a square. Four instances clear the KP4 threshold; the cheapest of them matches the recurrent form's bit-error rate at $1/59$ its area. The trend runs the wrong way from intuition: larger maps, to the right, are worse. Section VIII explains why.

VIII. TWO CORRECTIONS TO THE COST MODEL

A. Area tracks occupancy, not address width

The natural way to cost a map over a BW -bit address is 2^{BW} stored bits, and on that model the $B = 5$, $W = 4$ instance would hold a million bits. It synthesizes to 3537 gates. The reason is visible in the table: only 14 830 of 1 048 576 addresses are ever visited, 1.4 %, and logic minimization is free to choose any behaviour on the rest. Across all eight instances the measured gate count per occupied entry is

$$\text{gates} \approx 0.24 \times (\text{occupied entries}), \quad \text{spread } 0.15 \text{ to } 0.33, \quad (7)$$

while gates per *address* varies over two orders of magnitude, from 0.10 at eight address bits to 0.0034 at twenty. The correct planning quantity is therefore the number of distinct input patterns the map will actually see—which is bounded by the training set and is knowable in advance—not the address width. We emit the map as a truth table and leave the minimization to synthesis rather than instantiating memory, and (7) is what that costs.

A caution carried over from the analog measurements. The same mechanism that makes the map cheap can make it meaningless. An earlier identity-valued map in this flow mapped to $175 \mu\text{m}^2$ and 7 gates because the synthesizer resolved it to wires, while the same structure holding real contents mapped to $896 \mu\text{m}^2$ and 112 gates. Any area reported for an eliminated recurrence must be measured with trained contents, and any suspiciously small result must be checked against Step 1 of the method before it is believed. Our own 7-gate false positive in Section V is the cautionary case.

B. Widening the address makes the design worse

Reading Table II down the page, every quantity that intuition says should improve with a larger map gets worse: disagreement rises from 0.018 % to 0.840 %, bit-error rate from 8.89×10^{-5} to 8.33×10^{-3} , and area from 142.6 to 22 058.7 μm^2 . A hundredfold degradation in accuracy for a 155× increase in area is not a trade-off; it is a failure mode.

The mechanism is occupancy. At eight address bits, 112 of 256 entries (44 %) are trained from 240 000 symbols, so each carries thousands of observations. At twenty bits, 14 830 of 1 048 576 (1.4 %) are trained, each from a handful, and every unvisited address must fall back to a default that is wrong roughly half the time it is reached. **The binding resource is training data, not address space.** Proposition 1 accounts for the first two of the three error terms— $\rho(W)$ falls as W grows and $\epsilon_q(B)$ falls as B grows—but both are purchased with address bits, and a third term that the proposition omits, the unvisited-entry rate, rises faster. The transformation therefore has an interior optimum,

which is an unusual and useful property: the designer is looking for the *smallest* (B, W) that meets the tolerance, and enlarging it "for safety" is actively harmful. We have not yet written the third term in closed form; doing so, and predicting the optimum rather than finding it by sweep, is the first item in Section X.

C. Why the threshold, and not a numerical distance, is the metric

A substitute errs by letting stale state through, and stale state in a feedback loop produces *runs* of wrong decisions rather than isolated ones. Whether that matters depends on the outer code, and it matters enormously. For $RS(n, k)$ over $GF(2^m)$ with $t = (n-k)/2$, the post-decoding tail folds by $jC(n, j) = nC(n-1, j-1)$ to

$$\sum_{j>t} (j/n) C(n, j) p^j (1-p)^{n-j} = p \Pr\{\text{Bin}(n-1, p) \geq t\}, \quad (8)$$

and bisecting (8) for $RS(544, 514)$, $m = 10$, $t = 15$ at a 10^{-15} target gives the pre-decoding threshold 2.2619×10^{-4} used throughout, within 6 % of the value quoted in practice. Holding the pre-decoding bit-error rate *exactly* at that threshold and changing only the clustering gives Table III.

TABLE III

Post-decoding rate at a pre-decoding rate pinned to the KP4 threshold. Only the error structure differs. Beyond $t = 15$ symbols a single burst exhausts the code, so the last rows are arithmetic rather than modelling.

structure	events/codeword	post-FEC BER
random (assumed)	—	1.0e-13
bursts of 2 symbols	0.1230	1.7e-14
bursts of 4	0.0615	8.4e-9
bursts of 8	0.0308	6.8e-6
bursts of 16	0.0154	2.262e-4
bursts of 40	0.0062	2.262e-4

More than ten orders of magnitude separate designs that a mean-squared-error objective cannot distinguish and a pre-decoding bit-error-rate objective rates as identical. That the application metric should be preferred to a numerical one is standard advice [18]; the specific consequence for synthesis is that *the* application metric usually reached for is itself insufficient for this class of hardware. We therefore report error positions rather than only rates, so that the post-decoding evaluation is available, and we state plainly that the insufficiency of pre-decoding bit-error rate is known in the standards and instrumentation communities [17], [21], [22]—what is not established is its consequence for a synthesis objective function.

IX. REGISTER-TRANSFER IMPLEMENTATION

All three forms of Fig. 1 are given below in synthesizable SystemVerilog, with the dependence made explicit. The point of printing them is that the difference between them is visible in four lines, and so is the reason a scheduler cannot produce the third from the first.

(A) RECURRENT — THE CYCLE A SCHEDULER CANNOT BREAK

```

module dfe_recurrent #(parameter int K=8, XW=12, CW=8)
(input logic clk, rst_n, input logic signed [XW-1:0] x, output log
localparam logic signed [CW-1:0] C [K] = '{ /* adapted taps */ };
logic dly [K];
// past decisions, +-1 coded
logic signed [XW+CW:0] fb, u;
always_comb begin
// no multiplier: d is +-1, s
fb = '0;
// an add/subtract tree over
for (int k=0; k<K; k++) fb += dly[k] ? C[k] : -C[k];
end
assign u = (x << CW) - fb;
assign d = ~u[XW+CW]; // slicer = sign bit
always_ff @(posedge clk or negedge rst_n)
if (!rst_n) for (int k=0; k<K; k++) dly[k] <= '1'b0;
else begin dly[0] <= d; for (int k=1; k<K; k++) dly[k] <= dly[k-1]
endmodule
// The path x -> fb -> u -> d -> dly[0] closes within one clock. A r
// placed anywhere on it delays d past its own use at the next symbol
// RecMII >= depth(adder tree) + depth(slicer). This is the cycle.

```

(B) SPECULATIVE — CYCLE SHORTENED TO A MULTIPLEXER

```

module dfe_spec #(parameter int K=8, XW=12, CW=8)
(input logic clk, rst_n, input logic signed [XW-1:0] x, output log
localparam logic signed [CW-1:0] C [K] = '{ /* adapted taps */ };
logic dly [K];
logic signed [XW+CW:0] rest, u_p, u_m;
always_comb begin
// taps 2..K only: tap 1 is s
rest = '0;
for (int k=1; k<K; k++) rest += dly[k] ? C[k] : -C[k];
end
assign u_p = (x << CW) - (rest + C[0]); // branch: previous dec
assign u_m = (x << CW) - (rest - C[0]); // branch: previous dec
assign d = dly[0] ? ~u_p[XW+CW] : ~u_m[XW+CW]; // select, then
always_ff @(posedge clk or negedge rst_n)
if (!rst_n) for (int k=0; k<K; k++) dly[k] <= '1'b0;
else begin dly[0] <= d; for (int k=1; k<K; k++) dly[k] <= dly[k-1]
endmodule
// Both branches are computed before dly[0] is needed, so the cycle
// contains only the multiplexer. Cost: the summing path is duplicat
// per speculated step, i.e. |A|^d copies.

```

(C) ELIMINATED — NO CYCLE AT ALL

```

module dfe_elim #(parameter int B=4, W=2, XW=12, SH=5)
(input logic clk, rst_n, input logic signed [XW-1:0] x, output log
// 1. quantize: arithmetic shift and clamp only -- no multiplier
logic signed [XW-1:0] s; logic [B-1:0] q;
assign s = x >>> SH;
assign q = (s > ((1<<(B-1))-1)) ? {B{1'b1}} :
(s < -(1<<(B-1))) ? {B{1'b0}} :
B'(s + (1<<(B-1)));
// 2. window of W quantized inputs -- a plain shift register, forw
logic [B-1:0] hist [W-1:0];
always_ff @(posedge clk or negedge rst_n)
if (!rst_n) for (int i=0; i<W; i++) hist[i] <= '0;
else begin hist[0] <= q; for (int i=1; i<W; i++) hist[i] <= hist[i-1]
end
// 3. the trained map, emitted as a truth table; synthesis minimiz
logic [B*W-1:0] addr;
always_comb begin addr = '0;
for (int i=0; i<W; i++) addr[i*B +: B] = hist[i]; end
always_comb unique case (addr)
8'd0 : d = '1'b0; 8'd1 : d = '1'b0; /* ... 256 trained rows
default: d = '1'b1;
endcase
endmodule
// Every path runs forward. Registers may be inserted between the qu
// the window and the map at will, so II = 1 at whatever clock the m
// logic depth permits. Measured: 26 gates, 142.6 um2 at B=4, W=2.

```

The HLS form. In C the same three appear as a loop whose `dly` array is loop-carried (a), a loop with a speculated branch and a select (b), and a loop with no carried dependence at all (c). We expect a tool to report an initiation interval greater than one for (a) with the carried dependence named, a reduced but still non-unit interval for (b), and $II = 1$ for (c); confirming that, with the tool version recorded and a feed-forward filter as a positive control, is Stage 3 of Section X. We do not claim it here, because we have not run it.

X. RESEARCH PLAN

Each stage states the experiment, the platform, the controls that make it interpretable, and the outcome that would end the line of work.

A. Stage 1 — Close the model

Experiment. Write the third error term of Section VIII-B in closed form. The unvisited-entry rate for N training symbols over an address space of 2^{BW} is governed by the empirical occupancy distribution of the quantized window, which is a function of the slicer-input distribution and the tap vector. Predict the optimum (B, W) from ρ, ϵ_q and that term, and compare against a full sweep. **Control.** The prediction must be made and recorded before the sweep is run. **Kill condition.** If the predicted optimum is wrong by more than one step in either B or W on three of four operating points, the model does not size the transformation and the method reduces to an empirical sweep—useful, but not what is claimed here.

B. Stage 2 — Generalize across operating points and seeds

Experiment. Repeat Sections VI–VII on a grid of channel loss (15–30 dB), signal-to-noise ratio spanning the threshold, tap counts $K \in \{4, 8, 16\}$, and PAM4 as well as NRZ, with at least six independent seeds at each point and enough symbols for the statistical requirement below. **Controls.** (i) Step 1 of the method—the loop-gain check—applied at every point, with points where the loop is idle reported as excluded rather than silently averaged in. (ii) Seeds reported individually; a claim is made only where it holds on every seed. (iii) A linear-channel control, where the loop has nothing to do and the substitute must therefore *not* win; a method that wins everywhere is leaking. **Kill condition.** If the area advantage does not survive to $K = 16$ or to PAM4, the transformation is specific to short binary loops and must be described that way.

C. Stage 3 — High-level synthesis

Experiment. Describe every kernel of Table I in C, synthesize at a fixed target period with a commercial tool, and record reported II, reported RecMII, resource estimate, and the scheduling report's stated cause. Repeat after the source-level transformation. **Controls.** Identical pragmas, target period and tool version in both arms; a feed-forward filter of the same length as a positive control, which must reach II = 1 in both arms; explicit recording of the tool version, since scheduling behaviour is version-dependent and an unversioned II claim is not reproducible. **Kill condition.** Any Table I kernel the tool already pipelines to II = 1 without the transformation leaves the study, and we say which.

D. Stage 4 — FPGA measurement

Simulation cannot show that a transformed design closes timing, and it cannot reach the error rates the Section VIII-C argument needs. Both require hardware.

Device. A part with hard DSP slices and block RAM, so that the multiplier-free and table-based arms are priced on a fabric where multipliers are *not* free—the opposite of the platform bias that makes lookup methods look good on a fabric without them. A mid-range 16 nm-class device (Zynq UltraScale+ or Agilex class) suffices; the claim concerns II and relative resource, not an absolute frequency record. Block RAM capacity sets the practical ceiling on BW and must be reported as a constraint, not omitted: at $B = 4, W = 2$ the map is 256 entries and fits in logic, while at $B = 5, W = 4$ it is 2^{20} .

Harness. On-chip pattern generation, the datapath under test, and an on-chip comparator, all in the same clock domain. Off-chip capture cannot sustain the required bit counts and is not used for error statistics.

What is instrumented. Not merely a bit-error counter. Section VIII-C is precisely the argument that a bit-error counter is insufficient, so the harness must provide (i) a per-codeword symbol-error histogram over 544-symbol codewords, (ii) an error-position stream, decimated if necessary, for offline post-decoding evaluation, (iii) the burst-length distribution, and (iv) II and $f_{\max}$ as reported by place-and-route, not by synthesis estimate.

Statistical requirement. To separate a codeword-failure rate of 10^{-6} from 10^{-9} the rarer event must be seen tens of times, which at 5440 bits per codeword is of order 10^{11} bits per operating point. At a 100 MHz $\times$ 64-bit datapath that is a few minutes per point—feasible on-chip, impossible off-chip, and impossible in the software simulation this draft used. Operating points that do not reach the count are reported as bounds and labelled as such.

Controls. (i) A software reference must reproduce the hardware's error *positions*, not merely its rate, on a common seed and injected-noise sequence; if the positions differ, the two are not the same experiment and no comparison is made. (ii) Both arms at identical signal-to-noise ratio, seed and pattern. (iii) The recurrent arm measured on the same board in the same session, so device, temperature, toolchain and voltage are held fixed. (iv) A deliberately idle-loop configuration included as a negative control: there the substitute must reduce to a slicer and its area advantage must be reported as uninformative, which is the trap of Section V made into a standing check.

What is not measured, and is not claimed. An oscilloscope is used for clock integrity and timing-closure debug only; no analog claim is drawn from it. No radio-frequency, over-the-air, or anechoic measurement is involved, and the work makes no wireless claim. The standard-cell figures remain cell-area sums in a 130 nm library and are not post-place-and-route die area; they are to be read as ratios.

E. Stage 5 — The other kernels, including one expected to fail

Only the decision-feedback loop is measured here, and the remaining rows of Table I are the generalization test. Two are chosen deliberately. The add-compare-select recurrence forgets by survivor-path merging at a rate that is understood analytically, so it is the case where a *predicted* W can be checked against a measured one. The bang-bang timing loop is built to integrate and therefore to *not* forget; we expect ρ to decay far too slowly for any feasible window, and it is included as the honest negative example. A transformation with no negative example has not been characterized.

F. Deliverable

A source-level transformation, expressed as an HLS pragma with a supporting library, that takes a tolerance ϵ , measures ρ and ϵ_q from a calibration run, selects among recurrent, speculative and eliminated forms by the closed model of Stage 1, and reports the cost of its choice in both cell area and distance to the code's threshold.

XI. THREATS TO VALIDITY

- **Error counts are small.** On 360 000 held-out symbols, a bit-error rate of 8.89×10^{-5} is 32 errors and 2.22×10^{-4} is 80. The compliance calls nearest the threshold—in particular $B = 6, W = 2$ at 2.22×10^{-4} against a threshold of 2.2619×10^{-4} —are **not** statistically separated and should be read as "at the threshold", not as passing. The 58.9× area result does not depend on this, but the compliance column does.
- **One operating point, one seed.** Sections VI–VIII rest on a single channel, signal-to-noise ratio, tap set and seed. Stage 2 exists because of this.
- **Proposition 1 uses a single-step perturbation.** Whether flipping the most recent decision is the worst case over admissible state pairs is open; if it is not, ρ under-estimates and every window is optimistic.
- **The third error term is missing from the model.** Section VIII-B identifies the unvisited-entry rate as dominant at large addresses, and Proposition 1 does not contain it. The bound is therefore incomplete in exactly the regime where the design fails.
- **λ is fitted, not computed.** Equation (4) is used as a functional form; the substochastic spectral radius is not evaluated, because the input

process carries residual intersymbol interference and is not independent across symbols.

- *Areas are cell-area sums.* Post-place-and-route die area is larger by the utilization factor, and all drive strengths are the library minimum. Read as ratios only.
- *Channels are synthetic.* No measured scattering parameters were used.
- *Table III's event rates are a model.* The rows beyond $t = 15$ are arithmetic, but the mapping from burst length to event rate at fixed pre-decoding rate is modelled, and Stage 4 replaces it with measured positions. Interleaving, which standardized links use precisely to break bursts, is not yet modelled; whether an interleaver breaks the bursts a transformed loop produces is owed.
- *Bibliographic verification is incomplete.* Direct publisher access was unavailable from the environment in which this draft was prepared; entries identified through secondary sources are marked and must be checked before submission.

XII. CONCLUSION

The recurrence, not the arithmetic, sets the initiation interval of a receiver's inner loop, and the classical remedy stops at the quantizer every such loop contains. We removed one instead of shortening it, and sized the removal from a measurement rather than a constant. Modelled as an input-driven finite automaton, the loop forgets: two copies driven by a common input couple at a rate we measure at $\lambda = 0.338$ per symbol, so that a five-symbol window makes the earlier state irrelevant all but one time in 2500 — on a loop whose removal costs three orders of magnitude of bit-error rate. The resulting feed-forward substitute occupies $142.6 \mu\text{m}^2$ in 26 gates against $8395.6 \mu\text{m}^2$ in 924 for the recurrent form, at the same bit-error rate and inside the KP4 threshold, with no dependence cycle to schedule around. Two findings change how such a substitute should be planned: its area tracks the entries it actually occupies rather than its address width, so a million-entry map costs 3537 gates; and widening the address makes it worse, because the binding resource is training data. What remains is to close the third error term, to generalize across operating points, to confirm what a synthesis tool does with all three forms, and to measure on hardware at error rates software cannot reach. Section X says how, and says what would make us stop.

REFERENCES

1. K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. New York: Wiley, 1999. [STANDARD TEXT]
2. K. K. Parhi and D. G. Messerschmitt, "Pipeline interleaving and parallelism in recursive digital filters—Parts I and II," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 37, no. 7, pp. 1099–1134, Jul. 1989. [verify volume/pages]
3. G. Fettweis and H. Meyr, "Parallel Viterbi algorithm implementation: breaking the ACS-bottleneck," *IEEE Trans. Commun.*, vol. 37, no. 8, pp. 785–790, Aug. 1989. [verify pages]
4. A. Peled and B. Liu, "A new hardware realization of digital filters," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 22, no. 6, pp. 456–462, Dec. 1974. [verify pages]
5. A. Kavčić and J. M. F. Moura, "The Viterbi algorithm and Markov noise memory," *IEEE Trans. Inf. Theory*, vol. 46, no. 1, pp. 291–301, Jan. 2000. [CONFIRMED VIA INDEX]
6. E. Eleftheriou and W. Hirt, "Noise-predictive maximum-likelihood (NPML) detection for the magnetic recording channel," in *Proc. IEEE Int. Conf. Commun.*, 1996. [verify venue/year]
7. S. Kasturia and J. H. Winters, "Techniques for high-speed implementation of nonlinear cancellation," *IEEE J. Sel. Areas Commun.*, vol. 9, no. 5, pp. 711–717, Jun. 1991. [verify — canonical speculative-DFE reference]
8. A. A. Bazargani, H. Shakiba, and D. A. Johns, "MMSE equalizer design optimization for wireline SerDes applications," *IEEE Trans. Circuits Syst. I*, vol. 72, no. 4, pp. 1852–1865, 2025. [CONFIRMED VIA INDEX]
9. "COMPOSE: Static timing-driven composable reconfigurable architecture for accelerating recurrence-bound loops," arXiv:2606.21454. [verify authors; preprint]
10. J. Cheng, L. Josipović, G. A. Constantinides, P. Jenne, and J. Wickerson, "Combining dynamic and static scheduling in high-level synthesis" (DASS), in *Proc. ACM/SIGDA FPGA*, 2020. [verify year/pages]
11. J. Liu, J. Wickerson, and G. A. Constantinides, "Loop splitting for efficient pipelining in high-level synthesis," in *Proc. IEEE FCCM*, 2016. [CONFIRMED]
12. J. Liu, J. Wickerson, S. Bayliss, and G. A. Constantinides, "Polyhedral-based dynamic loop pipelining for high-level synthesis," *IEEE Trans. Comput.-Aided Design*, 2017. [CONFIRMED]
13. "Dynamic loop fusion in high-level synthesis," in *Proc. ACM/SIGDA FPGA*, 2025; arXiv:2501.14631. [CONFIRMED]
14. "A speculative loop pipeline framework with accurate path modeling for high-level synthesis," *ACM Trans. Reconfig. Technol. Syst.*, 2025, doi:10.1145/3705732. [CONFIRMED VIA DOI]
15. "FPGA HLS today: successes, challenges, and opportunities," *ACM Trans. Reconfig. Technol. Syst.*, doi:10.1145/3530775. [verify authors]
16. "ForgeHLS: A large-scale, open-source dataset for high-level synthesis," arXiv:2507.03255. [verify authors; preprint]
17. IEEE Std 802.3, Clause 91, Reed–Solomon forward error correction RS(544,514). [STANDARD]
18. "Approximate computing in high-level synthesis: from survey to practical implementation," *ACM Comput. Surv.*, doi:10.1145/3785334. [verify authors]
19. Y. Herklotz, Z. Du, N. Ramanathan, and J. Wickerson, "An empirical study of the reliability of high-level synthesis tools," in *Proc. IEEE FCCM*, 2021. [verify]
20. Y. Herklotz and J. Wickerson, "Finding and understanding bugs in FPGA synthesis tools," in *Proc. ACM/SIGDA FPGA*, 2020. [verify]
21. Keysight Technologies, "Relief on the search of burst errors," technical note, Jan. 2021. [trade source — replace with peer-reviewed equivalent if one exists]
22. D. Ofelt, "Pre-FEC BER monitoring proposal," IEEE P802.3bs contribution, Jan. 2016; X. Wang, "FEC performance over PAM4 links with bit-multiplexing," IEEE P802.3bs contribution, Jul. 2015. [verify contribution numbers]
23. Optical Internetworking Forum, *Next Generation CEI-448G Framework Document*, OIF-FD-CEI-448G-01.0, 2025. [verify document number]
24. SkyWater Technology and Google, *SkyWater Open Source PDK (sky130)*. [TOOL]
25. C. Wolf, Yosys *Open Synthesis Suite*; and R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Proc. CAV*, 2010. [TOOLS]
26. G. Ungerboeck, "Adaptive maximum-likelihood receiver for carrier-modulated data-transmission systems," *IEEE Trans. Commun.*, vol. 22, no. 5, pp. 624–636, May 1974. [verify pages]
27. A. Duel-Hallen and C. Heegard, "Delayed decision-feedback sequence estimation," *IEEE Trans. Commun.*, vol. 37, no. 5, pp. 428–436, May 1989. [verify pages]
28. M. V. Eyuboglu and S. U. H. Qureshi, "Reduced-state sequence estimation with set partitioning and decision feedback," *IEEE Trans. Commun.*, vol. 36, no. 1, pp. 13–20, Jan. 1988. [verify pages]